

DOCUMENTACIÓN INTERNA

YT-AGENT-AI — Reporte de Contenido

App web que analiza videos de YouTube: transcribe el audio con **AssemblyAI** y genera un **reporte de contenido estructurado** con **Google Gemini**. Interfaz **Streamlit** + API REST **Django**.

gemini-3.5-flash-lite

POST /generate-report/

Auth + Throttling

Docker

Descripción general

YT-AGENT-AI nació para traducir letras de canciones y evolucionó a un nuevo propósito: al ingresar una URL de YouTube, descarga el audio, lo transcribe y produce un reporte de contenido con 5 secciones (temas, argumento, opiniones, datos clave y conclusión).

PROPÓSITO ACTUAL

URL de YouTube → descarga de audio → transcripción (AssemblyAI) → reporte estructurado (Gemini) → persistencia en PostgreSQL → Streamlit / REST.

API KEYS

La clave de AssemblyAI (AAI_API_KEY) vive en el servidor vía `.env`. La clave de Gemini la provee cada usuario en el sidebar de Streamlit o en el payload REST (`gemini_api_key`); no va en el `.env`.

Flujo de ejecución

Entrada. Usuario provee URL de YouTube y su API Key de Gemini.

Descarga. `YouTubeService` obtiene título y descarga video/audio a `media/` con `yt-dlp`.

Transcripción. `TranscriptionService` envía el audio a `AssemblyAI` y guarda el texto.

Análisis. `AnalysisService` fragmenta la transcripción (chunks de 8000, máx. 30000) y genera el reporte con Gemini, con fallback entre modelos por cuota.

Persistencia. Resultado guardado en `translationPost` vía Django ORM.

Visualización. Reporte en Streamlit con botón de descarga en PDF, más video/audio.

Stack tecnológico

Backend	Django	4.1
Frontend	Streamlit	1.33.0
Transcripción	AssemblyAI	0.36.0
IA	Google Gemini	gemini-3.5-flash-lite
Descarga	yt-dlp	2025.9.26
PDF	fpdf2	2.7.9
API framework	Django REST Framework	3.14.0
Base de datos	PostgreSQL (Neon Cloud)	—
Contenedores	Docker + Compose	3.8

Estructura de directorios

```

Backend/
├── app.py                # Streamlit UI + generate_report_pdf()
├── manage.py             # Punto de entrada Django
├── cleanup_media.py      # Limpieza de temporales (>5 min)
├── crontab               # Cron cada 5 min
├── requirements.txt      # google-generativeai, fpdf2,
djangoRESTframework
├── .env.example          # plantilla de variables de entorno
├── translation_generator_app/
│   ├── services/         # youtube, transcription, analysis
│   ├── serializers/      # validación del payload
│   └── views/            # ContentAnalysisView, LoginView,
LogoutView
│   ├── exceptions.py     # AnalysisException, etc.
│   ├── models.py         # translationPost
│   └── urls.py           # /generate-report/
├── ai_translation/       # settings, urls, wsgi, asgi
├── media/               # videos/audio temporales
└── docs/                # ARCHITECTURE, KEY_COMPONENTS,
                        # GIT_REORGANIZATION_MANUAL,
                        # PROYECTO_STATE, docs.html

```

Componentes principales

Servicios

YOUTUBESERVICE

Extracción de título y descarga con yt-dlp (headers anti-bot).

```
get_title    download_video_and_audio
```

TRANSCRIPTIONSERVICE

Sube audio a AssemblyAI y devuelve texto crudo.

```
transcribe_audio
```

ANALYSISERVICE

Reporte de 5 secciones con Gemini + fallback por cuota.

```
generate_report    _try_with_fallback    _chunk_text
```

Interfaz Streamlit (app.py)

Contenedor ligero sin lógica de negocio: importa servicios directamente, usa `st.session_state`, captura excepciones personalizadas y ofrece descarga del reporte en PDF vía `generate_report_pdf()`.

Excepciones

BASE

`TranslationGeneratorException`

ESPECÍFICAS

`YouTubeDownloadException` `TranscriptionException` `AnalysisException`

`InvalidDataException`

Base de datos

Modelo `translationPost`: `youtube_title`, `youtube_link`, `generated_content` (reporte), `created_at`. Escritura al final del proceso exitoso.

Reporte de contenido (5 secciones)

Temas principales tratados.

Argumento / resumen narrativo.

Opiniones o puntos de vista.

Datos o hechos clave.

Conclusión / veredicto final.

API REST

/generate-report/

JSON — REQUEST

```
{
  "link": "https://youtube.com/watch?v=...",
  "gemini_api_key": "AI..."
}
```

JSON — RESPONSE 200

```
{
  "report": "Reporte de contenido...",
  "title": "Título del Video",
  "original_transcription": "Texto original...",
  "transcription_file": "/ruta/al/transcript.txt"
}
```

Autenticación

/login/

/logout/

BASH — LOGIN CON CURL

```
curl -c cookies.txt -X POST http://localhost:8000/login/ \
  -H "Content-Type: application/json" \
  -d '{"username": "usuario", "password": "secreto"}'

curl -b cookies.txt -X POST http://localhost:8000/generate-report/ \
  -H "Content-Type: application/json" \
  -d '{"link": "https://youtube.com/watch?v=...", "gemini_api_key":
  "AI..."}'
```

Errores

200	Login, logout o reporte generado
400	JSON inválido o validación del payload
401	Sin sesión o credenciales inválidas
405	Método distinto de POST
429	Throttling excedido (ver Seguridad)

401 Sin autenticación

```
{ "error": "Authentication required. Log in via POST /login/." }
```

429 Throttling excedido

```
{ "detail": "Request was throttled. Expected available in 60 seconds." }
```

Autenticación y throttling

Usuarios Django

El superusuario se crea una vez con `python manage.py createsuperuser`. Los usuarios adicionales se crean en `/admin/ → Users → Add user`. Los mismos usuarios sirven para Streamlit (pantalla de login con `authenticate()`) y para la API (sesión).

Throttling DRF

`rest_framework.throttling.ScopedRateThrottle` como clase por defecto en `ai_translation/settings.py` (`REST_FRAMEWORK`). Cada vista declara su `throttle_scope`:

LOGINVIEW → SCOPE LOGIN

Anti fuerza bruta, default 5/min.

LOGIN_THROTTLE_RATE

CONTENTANALYSISVIEW → SCOPE REPORT

Operación costosa, default 10/hour.

REPORT_THROTTLE_RATE

AJUSTE POR ENTORNO

Tasas en formato <n>/<s | m | h | d> vía .env, sin tocar código. Las vistas usan SessionNoCSRF (sesión Django sin exigir token CSRF) para que clientes no-navegador usen la cookie.

Variables de entorno

.env.example es la plantilla: AAI_API_KEY y DB_* requeridos; SECRET_KEY / DEBUG con default; LOGIN_THROTTLE_RATE / REPORT_THROTTLE_RATE opcionales. La API key de Gemini nunca va en el .env .

Cambios recientes

Propósito	Traducir canciones	Reporte de contenido
IA	OpenAI	Google Gemini
Endpoint	/api/generate-translation/	/generate-report/
Servicio	TranslationService	AnalysisService
Salida	Versos traducidos	Reporte + PDF

FALLBACK POR CUOTA

gemini-3.5-flash-lite → gemini-3.1-flash-lite → gemini-2.5-flash-lite → gemini-2.5-flash → gemini-3-flash → gemini-3.7-flash → gemini-3.8-flash → gemini-3.5-flash → gemini-3.6-flash → gemini-flash-latest → gemma-4-26b / gemma-4-31b. Retries ante 429/RESOURCE_EXHAUSTED y 404.

Configuración del entorno

```
# AssemblyAI - transcripción (servidor, nunca se pide al usuario)
AAI_API_KEY="..."

# NOTA: la API key de Google Gemini NO va en el .env.
# Cada usuario la ingresa en el sidebar de Streamlit
# o en el payload REST (gemini_api_key).

# Base de Datos (Neon Cloud PostgreSQL)
DB_NAME=neondb
DB_USER=neondb_owner
DB_HOST=...

# Django
SECRET_KEY=...
DEBUG=True
```

Despliegue

```
docker-compose up --build          # ambos servicios
docker compose build --no-cache backend # tras cambios en requirements
docker compose restart backend frontend # tras cambios de código
```

Frontend (Streamlit): <http://localhost:8501> · Backend API:

<http://localhost:8000> . Ambos montan `./media` ; solo el backend corre `crond` .

Mantenimiento

`cleanup_media.py` borra archivos de `media/` con más de 5 minutos (`AGE_LIMIT_SECONDS = 300` , por `mtime`). El cron lo ejecuta cada 5 minutos:

```
* /5 * * * * python /backend/cleanup_media.py >> /var/log/cron.log 2>&1CRON
```

NOTA

Tiempo real de borrado: 5–10 min tras la creación. Requiere el contenedor `backend` activo (es el único que inicia `cron`).

Preguntas frecuentes

¿Qué modelo de Gemini se usa?

Primario `gemini-3.5-flash-lite` (500 RPD) con cadena de fallback mapeada por cuota.

¿Por qué el reporte sale en español aunque el video sea en otro idioma?

El prompt instruye responder exclusivamente en español; no hay traducción, solo análisis.

¿Por qué falla con “video muy largo”?

La transcripción se trunca a 30000 caracteres y se procesa en chunks; si Gemini excede el timeout o la cuota diaria, se sugiere un video más corto o reintentar.

¿Para qué sirve la API key del .env si Gemini se pide en el frontend?

En el `.env` solo vive `AAI_API_KEY` (AssemblyAI), porque la transcripción nunca se pide al usuario. Gemini sí se pide por usuario.

¿Dónde está el historial Git?

Ver `docs/GIT_REORGANIZATION_MANUAL.md` (main reemplazada por feature/deploy, respaldo en main-old).

Checklist de verificación

`AnalysisService` importa en backend y frontend.

`google-generativeai==0.8.4` y `fpdf2==2.7.9` instalados en imágenes.

Endpoint `/generate-report/` responde (401 sin sesión).

Login/logout por sesión funcionando; throttling responde `429` al exceder la tasa.

Botón **Download Report (PDF)** visible tras generar el reporte.

`GEMINI_API_KEY` fuera del `.env` ; documentado en README y `PROYECTO_STATE`.

PDF pandoc (`PROYECTO_STATE.pdf`) y PDF diseñado generados.